

E4-TP2 : CODE MORSE

Prérequis :

- E4-Mémento1 : Pseudocode – Python – C/C++
- E2 : Algorithmique

Matériels et documentation :

- VS Code + PlatformIO
- Carte nodeMCU ESP32

Durée :**4H00****Problématique****Comment traduire des algorithmes en langage C++ pour les microcontrôleurs ESP32 ?****Objectifs**

À la fin de la séance, vous devrez être capable de :

- Écrire des codes en C++ permettant de programmer des ESP32
- Implémenter des tests et des boucles algorithmiques en C/C++
- Implémenter des fonctions en C/C++

On vous demande d'écrire et de traduire des algorithmes en C++ avec le *framework* Arduino permettant de réaliser des tests unitaires afin de réaliser l'émission d'un code morse.

1 ACTIVITÉ 1 : DÉCOUVERTE DU LANGAGE C++ ET DU FRAMEWORK ARDUINO

Arduino est un environnement open-source qui simplifie la création de projets électroniques. Il utilise une plateforme matériel-logiciel conviviale basée sur le langage de programmation **C/C++**.

Arduino offre une vaste bibliothèque de fonctions pré-écrites pour contrôler des composants électroniques, rendant le prototypage et le développement d'**applications embarquées** accessibles à tous les niveaux de compétence.

1.1 RÔLE DES FONCTIONS SETUP() ET LOOP()

- (01) Créer un projet nommé projet_2_esp32 avec **PlatformIO** sur Visual Studio Code. Écrire le code suivant dans le fichier **main.cpp**, le compiler et téléverser le binaire dans la carte nodeMCU ESP32. En déduire le rôle les différentes instructions en fonction du fonctionnement de la carte.

```
#include <Arduino.h>

const int8_t LED = GPIO_NUM_2;

void setup() {
    pinMode(LED, OUTPUT);
    digitalWrite(LED, HIGH);
    delay(5000);
}

void loop() {
    digitalWrite(LED, LOW);
    delay(400);
    digitalWrite(LED, HIGH);
    delay(400);
}
```



- (02) Expliquer le rôle des fonctions `setup()` et `loop()` du *framework* Arduino. Donner sous forme d'un algorithme l'appel de ces 2 fonctions par le microcontrôleur ESP32.

1.2 PROTOTYPE DE FONCTION

Le **prototype** de fonction est une déclaration qui décrit une fonction sans fournir son implémentation détaillée. Il spécifie le **type de retour** de la fonction, ainsi que les **types** et les **ordres des paramètres** qu'elle prend. Nous pouvons voir cela comme son **mode d'emploi simplifié**.

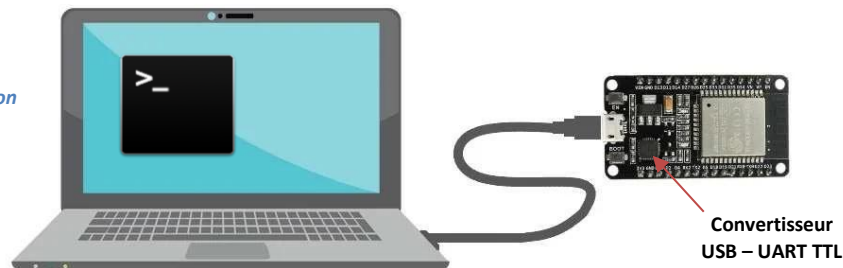
- (03) Relever le prototype des fonctions suivantes : `pinMode()`, `digitalWrite()` et `delay()`.
(04) Que représente le type `void` ?

1.3 COMMUNICATION SÉRIE PC ↔ NODEMCU ESP32

Pour réaliser une communication série entre la carte nodeMCU et le PC, nous allons utiliser le câble **USB** permettant l'alimentation et la programmation de la carte et le **terminal série** (disponible dans *platformIO*) sur le PC.

La carte nodeMCU dispose d'un composant (CP2102) permettant de convertir dans les deux sens le protocole **USB** (*Universal Serial Bus*) et le protocole **UART** (*Universal Asynchronous Receiver/Transmitter*).

Figure 1 : Principe de la communication série PC-nodeMCU ESP32



La communication UART étant asynchrone, il faudra veiller à programmer les mêmes vitesses de transmission sur les deux entités.

- (05) Compléter le fichier de configuration du projet *platformio.ini* pour régler la vitesse du terminal série du PC à 115 200 bauds. Un lien vers la documentation est donné dans le fichier.
(06) En utilisant les méthodes ci-dessous avec l'objet `Serial`. Modifier le fichier *main.cpp* précédent en ajoutant dans la fonction `setup()`, l'initialisation du port série et l'écriture du message « → fonction setup() » et dans la fonction `loop()` le message « → fonction loop() ». Compiler le code, téléverser le binaire et tester le fonctionnement en lançant le terminal série.

```
void HardwareSerial::begin(unsigned long baud, uint32_t config=SERIAL_8N1, ...)
```

Initialiser le port série avec une vitesse de transmission et si besoin une configuration spécifique

```
size_t Print::print(...)
```

Écrire une chaîne de caractère, la valeur d'une variable

```
size_t Print::println(...)
```

Écrire et revenir à la ligne

```
size_t Print::printf("...", ...)
```

Écrire une chaîne de caractère formatée

Syntaxe d'utilisation des méthodes précédentes avec l'objet `Serial` :

```
Serial.begin(...)
```

```
Serial.print(...)
```

```
Serial.println(...)
```

```
Serial.printf(...)
```

1.4 TYPAGE DES VARIABLES ET PLAGE DE VALEURS D'UN TYPE

Le langage C++ est fortement typé, c'est-à-dire que chaque variable doit être déclarée avec un type spécifique. Les types de base sont représentés dans les tableaux suivants.

Type booléen	Taille	Valeurs	Équivalence
bool	1 octet	false (0) ou true (1)	boolean

Type entier	Taille	Plage de valeurs	Équivalence
char	1 octet	-128 à +127	int8_t
unsigned char	1 octet	0 à 255	u_int8_t, byte
short	...	...	int16_t
unsigned short	...	...	u_int16_t
int	...	...	int32_t
unsigned int	...	...	u_int32_t, u_int
long	...	...	
unsigned long	...	...	
long long	...	...	int64_t
unsigned long long	...	...	u_int64_t

Type décimal	Taille	Plage de valeurs
float	...	...
double	...	...
long double	...	...

Type caractère	Taille	Valeurs
char	1 octet	Code ASCII étendu

Il faut garder à l'esprit que sur les microcontrôleurs, la mémoire est souvent limitée. Il faut donc utiliser un type adapté en fonction du contenu de la variable ; un type trop grand pourrait affecter la disponibilité de la mémoire.

L'opérateur `sizeof()` donne la taille en octets de son opérande. L'opérande peut être un type, une variable ou une expression.

- (07) Écrire le code en utilisant l'opérateur `sizeof()` pour afficher sur le terminal série la taille en octets des types de base. Vous calculerez ensuite la plage de valeurs de chaque type. Compléter les tableaux donnés.
- (08) Dans la fonction `setup()`, déclarer une variable `var` du type `short` et l'initialiser à 32 766. L'incrémenter 3 fois tout en affichant sa valeur dans le terminal série. Conclure sur les résultats obtenus.

1.5 PORTÉE D'UNE VARIABLE

- (09) Déclarer une variable `i` avec le type `int` dans la fonction `setup()`. Affecter cette variable avec la valeur 10 dans la fonction `loop()`, pourquoi y-a-t-il une erreur ?
- (10) Quels sont les possibilités pour corriger cette erreur ? Pour chaque possibilité vous donnerez les restrictions possibles.

2 ACTIVITÉ 2 : ALGORITHMES DU CODE MORSE

Le code Morse est un système de communication utilisant des combinaisons de points et de tirets pour représenter les lettres, chiffres et symboles, permettant la transmission de messages via des signaux sonores ou lumineux. Chaque caractère est codé par une séquence unique de points (courts signaux) et de tirets (longs signaux), avec des pauses entre les lettres et les mots pour une interprétation précise.

S • • •
O — — —
S • • •

2.1 MORSE – VERSION 1

- (11) Établir un algorithme pour réaliser un « SOS » en code Morse sur la LED de la carte nodeMCU ESP32. Vous utiliserez les recommandations suivantes :
 - Le « SOS » sera envoyé en boucle avec une temporisation de 2 secondes entre chaque émission,
 - La LED sera allumée pendant 0,25 seconde pour représenter un point et 0,75 seconde pour un tiret,
 - La LED sera éteinte pendant 0,25 seconde pour représenter un espace de séparation,
 - La LED sera éteinte pendant 0,5 seconde pour représenter une pause entre chaque lettre,
 - Ne pas utiliser de structures de contrôle (tests ou boucles) ou de fonctions.
- (12) Coder votre algorithme. Compiler le code et téléverser le binaire dans la carte. Tester le fonctionnement.
- (13) Comme toutes les durées sont multiples de 250 ms, vous remplacerez dans le code les durées de temporisation en utilisant une constante nommée TEMPO.

2.2 MORSE – VERSION 2

- (14) Modifier l'algorithme précédent en utilisant des boucles pour réaliser les répétitions de points ou de tirets.
- (15) Coder votre algorithme. Compiler le code et téléverser le binaire dans la carte. Tester le fonctionnement.

2.3 MORSE – VERSION 3

- (16) Modifier votre code en écrivant deux fonctions `point()` et `tiret()`.
- (17) Modifier votre code en écrivant une fonction `caractere(char lettre)`. Vous coderez au minimum les caractères 'S' et 'O'.

2.4 MORSE – BONUS

- (18) Modifier votre code en écrivant une fonction `morse(string texte)`. Vous testerez le code avec la chaîne "SOS".